claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a14d306a68c9a68b7.jsonl`
- SHA-256: `431de4809245955bb2e780e7e277ebc2589004a70ce993cc8ec3aecabb37809b`
- Source modified: `2026-06-10T16:27:40+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:25:50.931Z)

```
You are an adversarial verifier. A code reviewer audited the repo at
C:/Users/avidu/Projects/gmail-attachments-to-gdrive and reported this finding:

TITLE: Incremental sync advances historyId past messages that were not fully processed (silent
data loss)
SEVERITY CLAIMED: high
FILE: src/sync.ts (line 175-189)
DESCRIPTION: In the incremental branch, state.setHistoryId(changes.historyId) is executed
unconditionally after updateDrive returns. updateDrive deliberately does NOT mark a message
processed when the daily cap truncates it mid-message or when an attachment copy throws
(drive.ts:174-180), with the stated intent that 'the next run retries it'. But in incremental
mode the only retry mechanism is the history cursor, and it is advanced past those messages
regardless. The next run lists only changes AFTER changes.historyId, so cap-truncated and
transiently-failed messages are never seen again ? their attachments are silently never copied.
There is no fallback re-scan to catch them (the 30-day rewind only happens on cursor expiry).
The feature is off by default (ENABLE_INCREMENTAL_SYNC=false), but this is the designed steady
state once enabled.
EVIDENCE QUOTED: // Advance the cursor so the next run only sees newer changes.
        state.setHistoryId(changes.historyId);
...drive.ts: // If we stopped early on the daily cap (lastHandled short of the end) or
// hit a transient failure, leave it unmarked so the next run retries it.
SUGGESTED FIX: Only advance the cursor when the run completed cleanly: if updateDrive reported
errors or stopped at the cap (addedFiles == maxToAdd with messages left), keep the old historyId
(re-listing is cheap and dedup via processedSet + the filename probe makes overlap safe).
Alternatively, persist the unprocessed message ids in a small retry queue before advancing.

Your job is to try to REFUTE it. Your lens: correctness: trace the actual code path the finding
describes and check the claimed behavior is what the code really does (read the file and its
callers/tests).
Read the actual code in the repo (and tests/config where relevant). If after reading you cannot
confirm the problem is real, set isReal=false. If it is real but over/under-stated, set
adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/,
testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL
financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file
existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess
sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers,
balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account
number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.
Structured output only.
```

2. user (2026-06-10T16:25:57.156Z)

```
      import {
        ENABLE_INCREMENTAL_SYNC,
        MAX_FILES_ADD_PER_DAY,
        MAX_RUNTIME_MS,
        MAX_SYNC_TRIGGERS,
        ONE_DAY,
        ONE_MINUTE,
        SIX_HOURS,
        THIRTY_DAYS,
      } from "./config";
      import { updateDrive, type UpdateResult } from "./drive";
      import {
        getCurrentHistoryId,
        listAddedSince,
        threadsForMessageIds,
      } from "./gmailHistory";
      import { logDebug, logError } from "./logger";
      import { maybeSendDailySummary } from "./notify";
      import { addProcessedId } from "./processed";
      import { buildSearchQuery } from "./query";
      import { hasTime, isSameCalendarDay } from "./quota";
      import { dayKey, recordRun } from "./stats";
      import * as state from "./state";

      const SYNC_HANDLER = "syncUserDrives";

      /** Schedules a one-shot run of syncUserDrives `afterMs` from now. */
      export function newSyncTrigger(afterMs: number): void {
        ScriptApp.newTrigger(SYNC_HANDLER).timeBased().after(afterMs).create();
      }

      /** All currently-installed triggers for the sync handler. */
      function getSyncTriggers(): GoogleAppsScript.Script.Trigger[] {
        return ScriptApp.getProjectTriggers().filter(
          (t) => t.getHandlerFunction() === SYNC_HANDLER,
        );
      }

      /** Removes every project trigger (used when the user re-runs setup). */
      export function deleteAllTriggers(): void {
        for (const trigger of ScriptApp.getProjectTriggers()) {
          ScriptApp.deleteTrigger(trigger);
        }
      }

      function deleteTriggers(triggers: GoogleAppsScript.Script.Trigger[]): void {
        for (const trigger of triggers) {
          ScriptApp.deleteTrigger(trigger);
        }
      }

      /** Defense-in-depth: never let sync triggers exceed the cap (limit is 20). */
      function pruneSyncTriggers(max: number): void {
        const triggers = getSyncTriggers();
        for (let i = max; i < triggers.length; i++) {
          ScriptApp.deleteTrigger(triggers[i]);
        }
      }

      /**
       * The background worker. A per-user lock ensures only one run executes at a time
       * for THIS user, so concurrent triggers cannot stack file copies or trigger
       * generations. It is deliberately a user lock, not a script lock: the web app
```

```
 * runs as the accessing user (executeAs USER_ACCESSING), so a script-wide lock
 * would serialise every user's sync into one global queue.
 */
export function syncUserDrives(): void {
  const lock = LockService.getUserLock();
  if (!lock.tryLock(0)) {
    logDebug("Another sync run is in progress; skipping.");
    return;
  }
  try {
    runSyncOnce();
  } finally {
    // Bound the trigger count regardless of which path we took, then release.
    pruneSyncTriggers(MAX_SYNC_TRIGGERS);
    lock.releaseLock();
  }
}

/**
 * A single sync pass:
 *   1. reschedules a fan-out of backup triggers (self-healing if one is lost),
 *   2. syncs forward in 30-day windows until it runs out of time, hits the
 *      daily file cap, or fully catches up,
 *   3. persists progress and schedules the next run.
 */
function runSyncOnce(): void {
  const today = new Date();

  // On the first run of a new calendar day, summarise the previous day.
  maybeSendDailySummary(today);

  // Recover state if the debug flag was cleared (e.g. properties were wiped).
  const debug = state.getDebug();
  if (debug == null || debug === "0") {
    state.setDebug("1");
    state.setLastSynced(new Date(today.getTime() - 365 * ONE_DAY).toJSON());
  }

  // Create the backup triggers BEFORE deleting the current ones, so a failure
  // here can never leave us with no scheduled run.
  const previousTriggers = getSyncTriggers();
  try {
    newSyncTrigger(60 * ONE_MINUTE);
    newSyncTrigger(SIX_HOURS);
    newSyncTrigger(2 * SIX_HOURS);
    newSyncTrigger(4 * SIX_HOURS);
    newSyncTrigger(8 * SIX_HOURS);
    newSyncTrigger(28 * SIX_HOURS);
  } catch (e) {
    // Could not install new triggers; leave the old ones and try again later.
    logError("Could not install triggers: " + e);
    return;
  }
  deleteTriggers(previousTriggers);

  const filesAddedToday = state.getFilesAddedToday();
  let added = filesAddedToday.added;
  const lastCountDate = new Date(Date.parse(filesAddedToday.date));

  if (isSameCalendarDay(lastCountDate, today) && added >= MAX_FILES_ADD_PER_DAY) {
    logDebug("Used up all quota for today. Can't add more files.");
    newSyncTrigger(60 * ONE_MINUTE);
    return;
  }
  if (!isSameCalendarDay(lastCountDate, today)) {
```

```
        added = 0;
      }

      const structure = state.getFolderStructure();
      let lastSynced = state.getLastSynced();

      // Load the dedup set once; thread it through the run and persist at the end.
      let processedIds = state.getProcessedIds();
      const processedSet = new Set(processedIds);

      // This run's contribution to the day's stats.
      const addedAtStart = added;
      let runErrors = 0;
      let runLastError = "";

      // Folds one updateDrive result into this run's accumulators.
      const applyResult = (result: UpdateResult): void => {
        added += result.addedFiles;
        runErrors += result.errors;
        if (result.lastError) {
          runLastError = result.lastError;
        }
        for (const id of result.processedIds) {
          if (!processedSet.has(id)) {
            processedSet.add(id);
            processedIds = addProcessedId(processedIds, id);
          }
        }
      };

      try {
        // Incremental mode is opt-in (ENABLE_INCREMENTAL_SYNC) so it rolls out
        // gradually over the proven date-window backfill. A stored historyId only
        // exists once a backfill has caught up while the flag was on.
        const historyId = ENABLE_INCREMENTAL_SYNC ? state.getHistoryId() : null;
        if (historyId) {
          // Incremental: process only messages added since the stored historyId.
          const changes = listAddedSince(historyId);
          if (changes == null) {
            // History expired: clear the cursor and re-scan a bounded recent window
            // (the next catch-up re-captures a fresh historyId). Overlap is safe ?
            // dedup by message id and the Drive filename guard prevent re-copying.
            state.clearHistoryId();
            lastSynced = new Date(today.getTime() - THIRTY_DAYS).toJSON();
            logDebug("History expired; reverting to date-window sync.");
          } else {
            if (changes.messageIds.length > 0 && added < MAX_FILES_ADD_PER_DAY) {
              const threads = threadsForMessageIds(changes.messageIds);
              if (threads.length > 0) {
                applyResult(
                  updateDrive(
                    threads,
                    structure,
                    MAX_FILES_ADD_PER_DAY - added,
                    processedSet,
                  ),
                );
              }
            }
            // Advance the cursor so the next run only sees newer changes.
            state.setHistoryId(changes.historyId);
          }
        } else {
          // Backfill: walk forward in 30-day windows until caught up to today.
          // Capture the mailbox historyId BEFORE processing so mail arriving
```

```
          // mid-run is still picked up by the first incremental scan.
          const candidateHistoryId = ENABLE_INCREMENTAL_SYNC
            ? getCurrentHistoryId()
            : null;
          let fullSynced = false;
          while (
            hasTime(new Date(), today, MAX_RUNTIME_MS) &&
            added < MAX_FILES_ADD_PER_DAY &&
            !fullSynced
          ) {
            const maxToAdd = MAX_FILES_ADD_PER_DAY - added;
            const search = buildSearchQuery(lastSynced, new Date());
            logDebug("Query: " + search.query + "  files asking: " + maxToAdd);

            const threads = GmailApp.search(search.query);
            if (threads.length > 0) {
              applyResult(updateDrive(threads, structure, maxToAdd, processedSet));
            }

            fullSynced = search.isToday;
            if (added < MAX_FILES_ADD_PER_DAY) {
              lastSynced = fullSynced
                ? today.toJSON()
                : new Date(Date.parse(lastSynced) + THIRTY_DAYS).toJSON();
            }
          }
          // Caught up: switch future runs to the cheaper incremental path.
          if (candidateHistoryId && fullSynced && added < MAX_FILES_ADD_PER_DAY) {
            state.setHistoryId(candidateHistoryId);
            logDebug("Backfill complete; stored historyId " + candidateHistoryId);
          }
        }
      } catch (e) {
        logError("Some error in execution: " + e);
        state.setLastError(String(e));
        runErrors++;
        runLastError = String(e);
      }

      // Fold this run into the day's accumulator.
      const dayStats = recordRun(state.getRunStats(dayKey(today)), {
        filesAdded: added - addedAtStart,
        errors: runErrors,
        lastError: runLastError,
      });
      state.setRunStats(dayStats);

      state.setProcessedIds(processedIds);
      state.setLastSynced(lastSynced);
      state.setFilesAddedToday({ added, date: today.toJSON() });
      newSyncTrigger(20 * ONE_MINUTE);
    }

```

3. user (2026-06-10T16:25:58.197Z)

```
import { GMAIL_ATTACHMENT_FOLDER } from "./config";
import { isAllowedContentType } from "./contentTypes";
import { getAttachmentName, getFolderNameForStructure } from "./naming";
import { logDebug, logError } from "./logger";

type Folder = GoogleAppsScript.Drive.Folder;

/**
 * Anything that can hold folders: both `DriveApp` (the root) and a `Folder`
```

```
 * expose `getFolders()` / `createFolder()`, so we accept either structurally.
 */
interface FolderContainer {
  getFolders(): GoogleAppsScript.Drive.FolderIterator;
  createFolder(name: string): Folder;
}

export interface UpdateResult {
  /** Number of files actually written to Drive in this call. */
  addedFiles: number;
  hasError: boolean;
  /** Count of attachment-level errors in this call. */
  errors: number;
  /** Most recent error message, if any. */
  lastError: string;
  /** Gmail message ids fully handled in this call (for cross-run dedup). */
  processedIds: string[];
}

/** Returns the named child folder, creating it if it does not exist yet. */
export function getOrCreateFolderInFolder(
  parent: FolderContainer,
  name: string,
): Folder {
  const folders = parent.getFolders();
  while (folders.hasNext()) {
    const folder = folders.next();
    if (folder.getName() === name) {
      logDebug("Found existing folder " + name);
      return folder;
    }
  }
  logDebug("Creating new folder " + name);
  return parent.createFolder(name);
}

/**
 * Resolves the destination sub-folder for an attachment, creating it under
 * `head` and caching it in `existingFolders`. Returns null when the chosen
 * structure yields no folder (caller falls back to the head folder).
 */
function resolveFolder(
  head: Folder,
  existingFolders: Map<string, Folder>,
  structure: string,
  contentType: string,
  date: Date,
): Folder | null {
  const name = getFolderNameForStructure(structure, contentType, date);
  if (name === "") {
    return null;
  }
  const cached = existingFolders.get(name);
  if (cached) {
    return cached;
  }
  const created = head.createFolder(name);
  existingFolders.set(name, created);
  return created;
}

/**
 * Copies every allowed, not-yet-present attachment from the given threads into
 * the GmailAttachments folder, organised per the user's chosen structure.
 * Stops once `maxToAdd` files have been written.
```

```
75       *
76       * `alreadyProcessed` is the set of Gmail message ids handled in previous runs;
77       * such messages are skipped wholesale, avoiding a Drive `getFilesByName` probe
78       * per attachment when an overlapping date window is re-scanned. Ids fully
79       * handled here are returned in `processedIds` for the caller to persist.
80       */
81      export function updateDrive(
82        threads: GoogleAppsScript.Gmail.GmailThread[],
83        structure: string,
84        maxToAdd: number,
85        alreadyProcessed: ReadonlySet<string> = new Set(),
86      ): UpdateResult {
87        const head = getOrCreateFolderInFolder(DriveApp, GMAIL_ATTACHMENT_FOLDER);
88
89        // Pre-load existing sub-folders so we reuse rather than duplicate them.
90        const existingFolders = new Map<string, Folder>();
91        const currentFolders = head.getFolders();
92        while (currentFolders.hasNext()) {
93          const folder = currentFolders.next();
94          existingFolders.set(folder.getName(), folder);
95        }
96        logDebug("Already present folders: " + existingFolders.size);
97
98        const addedFiles = new Map<string, Folder>();
99        const processedIds: string[] = [];
100       let addMore = maxToAdd > 0;
101       let hasError = false;
102       let errors = 0;
103       let lastError = "";
104
105       for (const thread of threads) {
106         if (!addMore) break;
107         for (const message of thread.getMessages()) {
108           if (!addMore) break;
109           const messageId = message.getId();
110           // Skip messages we fully handled on a previous run.
111           if (alreadyProcessed.has(messageId)) {
112             logDebug("Skipping already-processed message " + messageId);
113             continue;
114           }
115           let messageFailed = false;
116           const from = message.getFrom();
117           // GmailMessage#getDate is typed as the stripped Base.Date interface but is
118           // a real JS Date at runtime.
119           const date = message.getDate() as unknown as Date;
120           const attachments = message.getAttachments();
121           let lastHandled = -1;
122           for (let a = 0; a < attachments.length && addMore; a++) {
123             const attachment = attachments[a];
124             lastHandled = a;
125             const contentType = attachment.getContentType();
126             const fileName = getAttachmentName(attachment.getName(), from, date);
127
128             if (!isAllowedContentType(contentType) || addedFiles.has(fileName)) {
129               logDebug("Rejected file of type: " + contentType);
130               continue;
131             }
132
133             let folder = resolveFolder(
134               head,
135               existingFolders,
136               structure,
137               contentType,
138               date,
139             );
```

```
            if (folder == null) {
              // Unknown structure: drop it in the top-level folder for recovery.
              logError("Could not resolve directory; using top-level folder.");
              folder = head;
            }

            try {
              // Skip if a file with this name already exists in the folder.
              if (folder.getFilesByName(fileName).hasNext()) {
                continue;
              }
              const blob = attachment.copyBlob();
              blob.setName(fileName);
              const file = folder.createFile(blob);
              file.setDescription(
                "Created from a Gmail Attachment from " +
                  from +
                  " sent on date " +
                  date,
              );
              addedFiles.set(fileName, folder);
              Utilities.sleep(100);
              if (addedFiles.size === maxToAdd) {
                addMore = false;
              }
            } catch (e) {
              logError("Attachment copy failed (message " + messageId + "): " + e);
              hasError = true;
              messageFailed = true;
              errors++;
              lastError = String(e);
              Utilities.sleep(1000);
            }
          }
          // Mark the message done only if we handled all its attachments cleanly.
          // If we stopped early on the daily cap (lastHandled short of the end) or
          // hit a transient failure, leave it unmarked so the next run retries it.
          const completed = !messageFailed && lastHandled === attachments.length - 1;
          if (completed) {
            processedIds.push(messageId);
          }
        }
      }

      return { addedFiles: addedFiles.size, hasError, errors, lastError, processedIds };
    }

```

4. assistant (2026-06-10T16:26:21.411Z)

The finding describes the incremental branch precisely. Let me verify the helper semantics in gmailHistory.ts, the config default, and any tests covering this.

5. user (2026-06-10T16:26:22.799Z)

```
      import {
        collectAddedMessageIds,
        isHistoryExpired,
        nextHistoryId,
        type HistoryListResponse,
      } from "./historySync";
      import { logDebug } from "./logger";

      type GmailThread = GoogleAppsScript.Gmail.GmailThread;

```

```
    /** A bounded number of history pages to walk in a single run. */
    const MAX_HISTORY_PAGES = 20;

    /** The mailbox's current historyId, used to seed incremental sync. */
    export function getCurrentHistoryId(): string | null {
      const profile = Gmail.Users.getProfile("me");
      return profile.historyId ?? null;
    }

    /** What an incremental scan yields: new message ids and the id to store next. */
    export interface IncrementalChanges {
      messageIds: string[];
      historyId: string;
    }

    /**
     * Lists messages added since `startHistoryId` via the Gmail advanced service,
     * paging through results. Returns null when the history has expired (the caller
     * must fall back to a full sync). Only `messageAdded` events are requested.
     */
    export function listAddedSince(
      startHistoryId: string,
    ): IncrementalChanges | null {
      const responses: HistoryListResponse[] = [];
      let pageToken: string | undefined;
      try {
        for (let page = 0; page < MAX_HISTORY_PAGES; page++) {
          const response = Gmail.Users.History.list("me", {
            startHistoryId,
            historyTypes: ["messageAdded"],
            pageToken,
          }) as HistoryListResponse;
          responses.push(response);
          if (!response.nextPageToken) {
            break;
          }
          pageToken = response.nextPageToken;
        }
      } catch (e) {
        if (isHistoryExpired(e)) {
          logDebug("History id expired; full sync required.");
          return null;
        }
        throw e;
      }
      return {
        messageIds: collectAddedMessageIds(responses),
        historyId: nextHistoryId(responses, startHistoryId),
      };
    }

    /**
     * Resolves the distinct Gmail threads containing the given message ids. Threads
     * are de-duplicated by id so a multi-message thread is processed once. Messages
     * that can no longer be fetched (deleted since the change was recorded) are
     * skipped.
     */
    export function threadsForMessageIds(messageIds: string[]): GmailThread[] {
      const threads = new Map<string, GmailThread>();
      for (const id of messageIds) {
        const message = GmailApp.getMessageById(id);
        if (!message) {
          continue;
        }
        const thread = message.getThread();
```

```
76          threads.set(thread.getId(), thread);
77        }
78        return Array.from(threads.values());
79      }
80
```

6. user (2026-06-10T16:26:23.874Z)

```
18- * Apps Script enforces a daily quota on file creation. We cap ourselves below
19- * it so a single run can never exhaust the account's allowance.
20- */
21:export const MAX_FILES_ADD_PER_DAY = 240;
22-
23-// Time constants, in milliseconds.
24-export const ONE_MINUTE = 60_000;
--
60- * Requires the Gmail advanced service (declared in appsscript.json); no extra
61- * OAuth scope ? `gmail.readonly` already covers history reads.
62- */
63:export const ENABLE_INCREMENTAL_SYNC = false;
64-
65-/**
66- * Daily summary email. OFF by default ? enabling it requires the
```

7. user (2026-06-10T16:26:24.195Z)

```
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\auditability.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\contentTypes.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\historySync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\naming.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notify.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\processed.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\quota.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\stats.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts
```

8. assistant (2026-06-10T16:26:28.925Z)

Let me check the incremental sync test and historySync helpers to confirm cursor semantics.

9. user (2026-06-10T16:26:29.811Z)

```
1       // Enable the opt-in incremental path for this suite only.
2       jest.mock("../src/config", () => ({
3         ...jest.requireActual("../src/config"),
4         ENABLE_INCREMENTAL_SYNC: true,
5       }));
6
7       import {
8         ONE_DAY,
9         PROP_DEBUG,
10        PROP_FILES_ADDED_TODAY,
11        PROP_FOLDER,
12        PROP_HISTORY_ID,
13        PROP_LAST_SYNCED,
14        PROP_PROCESSED_IDS,
15      } from "../src/config";
16      import { syncUserDrives } from "../src/sync";
17
```

```
type Store = Record<string, string>;
const g = globalThis as unknown as Record<string, unknown>;
const nowIso = () => new Date().toJSON();

function fakeProps(initial: Store) {
  const store: Store = { ...initial };
  return {
    store,
    getProperty: (k: string) => (k in store ? store[k] : null),
    setProperty: (k: string, v: string) => {
      store[k] = String(v);
    },
    deleteProperty: (k: string) => {
      delete store[k];
    },
    deleteAllProperties: () => {
      for (const k of Object.keys(store)) delete store[k];
    },
  };
}

function fakeScriptApp() {
  const triggers: { getHandlerFunction: () => string }[] = [];
  return {
    newTrigger: (handler: string) => ({
      timeBased: () => ({
        after: () => ({
          create: () => triggers.push({ getHandlerFunction: () => handler }),
        }),
      }),
    }),
    getProjectTriggers: () => [...triggers],
    deleteTrigger: () => undefined,
  };
}

function iter<T>(items: T[]) {
  let i = 0;
  return { hasNext: () => i < items.length, next: () => items[i++] };
}

function installBase(props: ReturnType<typeof fakeProps>) {
  g.PropertiesService = { getUserProperties: () => props };
  g.ScriptApp = fakeScriptApp();
  g.LockService = {
    getUserLock: () => ({ tryLock: () => true, releaseLock: jest.fn() }),
  };
  g.GmailApp = { search: jest.fn(() => []), getMessageById: jest.fn() };
}

function baseProps(overrides: Store = {}) {
  return fakeProps({
    [PROP_DEBUG]: "1",
    [PROP_FOLDER]: "byContentType",
    [PROP_LAST_SYNCED]: nowIso(),
    [PROP_FILES_ADDED_TODAY]: JSON.stringify({ added: 0, date: nowIso() }),
    ...overrides,
  });
}

describe("syncUserDrives ? incremental mode", () => {
  it("processes messages added since the stored historyId and advances it", () => {
    const props = baseProps({ [PROP_HISTORY_ID]: "100" });
    installBase(props);

```

```
        // A Drive head folder that accepts one file.
        const head = {
          getName: () => "GmailAttachments",
          getFolders: () => iter([]),
          createFolder: () => head,
          getFilesByName: () => iter([]),
          createFile: () => ({ setDescription: () => undefined }),
        };
        g.DriveApp = { getFolders: () => iter([]), createFolder: () => head };

        const message = {
          getId: () => "m1",
          getFrom: () => "alice@example.com",
          getDate: () => new Date(2012, 5, 6),
          getAttachments: () => [
            {
              getContentType: () => "application/pdf",
              getName: () => "report.pdf",
              copyBlob: () => ({ setName: () => undefined }),
            },
          ],
        };
        (g.GmailApp as { getMessageById: jest.Mock }).getMessageById = jest.fn(
          () => ({ getThread: () => ({ getId: () => "t1", getMessages: () => [message] })
}),
        );
        g.Gmail = {
          Users: {
            getProfile: jest.fn(),
            History: {
              list: jest.fn(() => ({
                history: [{ messagesAdded: [{ message: { id: "m1" } }] }],
                historyId: "200",
              })),
            },
          },
        };

        syncUserDrives();

        // Never falls back to a date-window search while the history is valid.
        expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
        expect(props.store[PROP_HISTORY_ID]).toBe("200");
        expect(JSON.parse(props.store[PROP_PROCESSED_IDS])).toContain("m1");
      });

      it("clears the cursor and rewinds lastSynced when the history expired", () => {
        const props = baseProps({ [PROP_HISTORY_ID]: "1" });
        installBase(props);
        g.Gmail = {
          Users: {
            getProfile: jest.fn(),
            History: {
              list: jest.fn(() => {
                throw new Error("call failed with error: 404 not found");
              }),
            },
          },
        };

        syncUserDrives();

        expect(PROP_HISTORY_ID in props.store).toBe(false);
        // lastSynced rewound to roughly 30 days ago for the recovery re-scan.
        const lastSynced = Date.parse(props.store[PROP_LAST_SYNCED]);
```

```
        expect(Date.now() - lastSynced).toBeGreaterThan(29 * ONE_DAY);
        // No history ? no incremental processing this run; backfill resumes next run.
        expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
      });

      it("captures the mailbox historyId once backfill catches up", () => {
        // No stored historyId ? backfill mode; lastSynced=now means we are already
        // caught up, so the run captures the current historyId for next time.
        const props = baseProps();
        installBase(props);
        g.Gmail = {
          Users: {
            getProfile: jest.fn(() => ({ historyId: "555" })),
            History: { list: jest.fn() },
          },
        };

        syncUserDrives();

        expect((g.GmailApp as { search: jest.Mock }).search).toHaveBeenCalled();
        expect(props.store[PROP_HISTORY_ID]).toBe("555");
      });
    });

```

10. user (2026-06-10T16:26:30.232Z)

```
    /*
     * Pure helpers for the Gmail `users.history.list` incremental sync. These
     * operate on plain data shapes (the JSON the Gmail advanced service returns)
     * and have no GAS dependencies, so they are exercised directly in unit tests.
     * The actual advanced-service calls live in the `gmailHistory` shell.
     *
     * See https://developers.google.com/workspace/gmail/api/guides/sync ? a stored
     * `startHistoryId` returns only the changes since that point (~2 quota units vs
     * ~5 for a list call). The id can expire, in which case the API responds 404
     * and the caller must fall back to a full (date-window) sync.
     */

    /** A message stub as it appears inside a history record. */
    export interface HistoryMessage {
      id?: string;
      threadId?: string;
    }

    /** One change record; we only care about messages added to the mailbox. */
    export interface HistoryRecord {
      messagesAdded?: { message?: HistoryMessage }[];
    }

    /** The shape of a `Gmail.Users.History.list` response we depend on. */
    export interface HistoryListResponse {
      history?: HistoryRecord[];
      historyId?: string;
      nextPageToken?: string;
    }

    /**
     * Collects the unique ids of messages added across the given history pages, in
     * first-seen order. Records without a `messagesAdded` entry (e.g. label-only
     * changes) contribute nothing.
     */
    export function collectAddedMessageIds(
      responses: HistoryListResponse[],
    ): string[] {
```

```
39        const seen = new Set<string>();
40        const ids: string[] = [];
41        for (const response of responses) {
42          for (const record of response.history ?? []) {
43            for (const added of record.messagesAdded ?? []) {
44              const id = added.message?.id;
45              if (id && !seen.has(id)) {
46                seen.add(id);
47                ids.push(id);
48              }
49            }
50          }
51        }
52        return ids;
53      }
54
55      /**
56       * The newest `historyId` to persist for the next run: the last page's id, or
57       * the supplied fallback (the start id) if no page reported one.
58       */
59      export function nextHistoryId(
60        responses: HistoryListResponse[],
61        fallback: string,
62      ): string {
63        for (let i = responses.length - 1; i >= 0; i--) {
64          const id = responses[i].historyId;
65          if (id) {
66            return id;
67          }
68        }
69        return fallback;
70      }
71
72      /**
73       * Whether an error thrown by `history.list` means the stored `startHistoryId`
74       * has expired. Gmail responds with HTTP 404 ("Requested entity was not found")
75       * once the id predates the mailbox's retained history; the caller must then do
76       * a full sync. We match defensively on both the status code and the message
77       * text since the advanced service surfaces errors as a stringified exception.
78       */
79      export function isHistoryExpired(error: unknown): boolean {
80        const e = error as { message?: string; details?: { code?: number } } | null;
81        if (e && e.details && e.details.code === 404) {
82          return true;
83        }
84        const text = String(e?.message ?? error ?? "").toLowerCase();
85        return text.includes("404") || text.includes("not found");
86      }
87
```

11. user (2026-06-10T16:27:35.812Z)

Structured output provided successfully

12. assistant (2026-06-10T16:27:40.628Z)

Verification complete. The finding is confirmed real: the incremental sync path in src/sync.ts advances the history cursor unconditionally, defeating drive.ts's deliberate retry mechanism for cap-truncated and transiently-failed messages, resulting in silent data loss once ENABLE_INCREMENTAL_SYNC is enabled.